Draw it or lose it

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4

Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	03/27/2025	Rachael Thrasher	First iteration

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

This software design document outlines our proposed approach to building the web-based game application for our client The Gaming Room for “Draw It or Lose It” a high-energy interactive team guessing game modeled after Win, Lose, or Draw but differing from the traditional version in several unique features. In this adaptation, stock drawings are used as clues and four rounds of action are played with simultaneous scoring of multiple teams. By creating a web-based version, this exciting game can be used on any platform and optimized for best user experience. The design strives to meet all of the requested features by The Gaming Room and results in an exciting, dynamic and straightforward interface for end users.

Requirements

1. The game must be available through a web-based platform so it can run on various devices and operating systems.
2. Each game should allow one or more teams to participate, with multiple players assigned to each team.
3. Game and team names must be unique to avoid conflicts, and users should be able to check name availability when creating a team.

4. The application should ensure only one instance of a game exists in memory at a time by using unique identifiers for games, teams, and players.
5. Each round should have a set time limit, such as one minute, with drawings gradually revealed and fully visible by the 30-second point.
6. If a team does not correctly guess the puzzle within the allotted time, other teams should be given one opportunity each to guess within a 15-second window.

Design Constraints

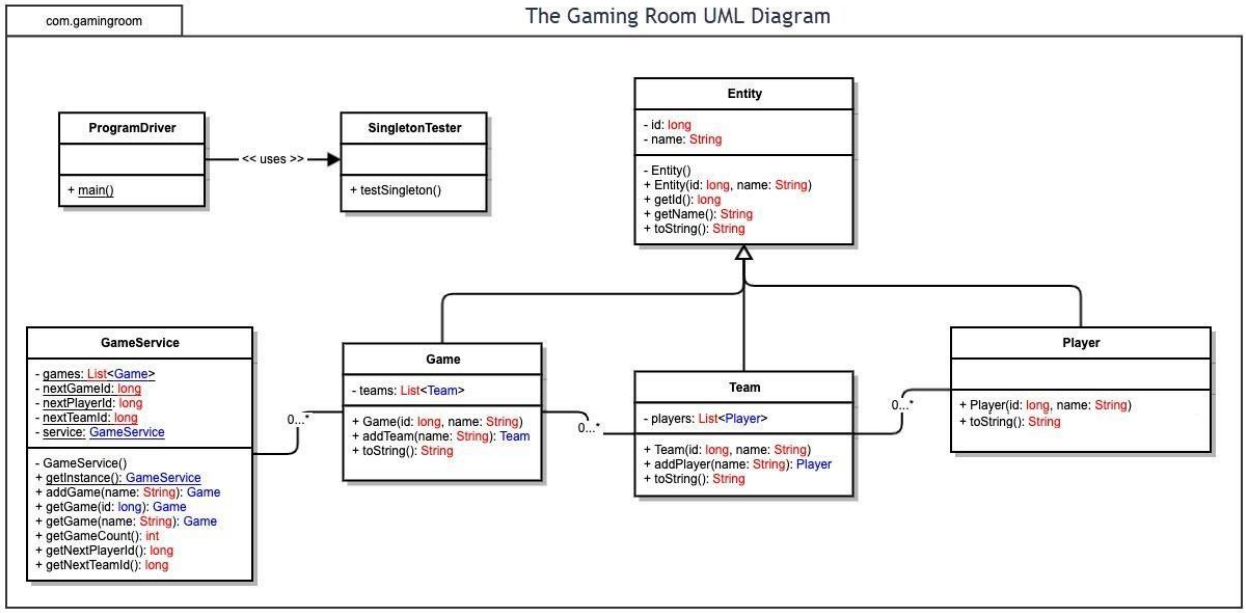
1. The game should be implemented for a web-based platform (web-based distributed environment) and hence be based on network communication, require appropriate security measures, and must work with different browsers and on a variety of hardware platforms.
2. Require unique names for games, teams and players so that when you are creating or joining a game, the experience is better.
3. The designed solution must be implemented in a way such that only one instance of the game service is running at the same time to make the application work correctly

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

<Describe the UML class diagram provided below. Explain how the classes relate to each other. Identify any object-oriented programming principles that are demonstrated in the diagram and how they are used to fulfill the software requirements efficiently.>



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	**Mac**	**Linux**	**Windows**	**Mobile Devices**

Server Side	Unix-based platforms are a tried-and-tested stable and secure base for webbased solutions with mature ecosystems and developer tools. However, Unix platforms generally demand higher hardware costs and lack the massive scalability offered by Linux and Windowsbased platforms.	Linux is an opensource operating system that gives users full control and customization capabilities, offering a wide array of software and utilities. Linux offers great scalability and is known for being a very stable and secure operating system. Although the command-line oriented interface can be somewhat of a challenge, Linux also offers a wide variety of graphical interfaces to choose from.	Windows has support for a wide range of software, a large and mature developers community, very good hardware support and an excellent set of documentation. Windows is also associated with a number of security vulnerabilities above other operating systems.	Touchscreen and gesture-based interaction with limited screen size. Varied hardware Capabilities for capabilities to include content. Capabilities can include more than one graphic object and can be linked to other capabilities, which themselves can be linked to additional capabilities and so on.
Client Side	The interface is intuitive and easy to use. However, the costs to the developer in creating and maintaining Lister, the Windows Client, the Web Client and the Mac Client are reflected in the longer than anticipated development period and the broad range of skills required to complete the project.	Although free to use and freely distributed, it is not free of all costs (e.g. hardware and development tools) and can be more complex to learn than traditional code and may require additional skills to generate code that runs on different client platforms.	Our licensing fees are higher than many opensource solutions.	Consider factors such as designing for multiple screen sizes, project constraints like internet connectivity, and using native device features like the camera, GPS, and push notifications.

Development Tools	Node.js, JavaScript, supports integration with popular development environments such as Visual Studio Code and Xcode	A healthy information ecosystem includes not only robust integrated development environments like VS Code, Atom and Sublime Text, but a powerful command line interface as well as robust information about packages and package managers like apt and yum.	C# and .NET is often associated with the development of Windows applications but can also be used to build web applications for the Windows operating system. This guide focuses on basic website development and covers how to get started with C# for web application projects using a variety of popular tools and environments.	Android Studio, Xcode, simulators and physical Android and iOS devices to support our Android and iOS projects. Languages used on these platforms include Kotlin, Swift, and Objective-C, in addition to Java and JavaScript.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**:
   The Gaming Room may benefit from having Draw It or Lose It available on an Operating Platform. Having a web based version would offer the ability to play on desktops, laptops, tablets and smartphones via a browser, reaching a greater demographic while delivering a seamless experience.

2. **Operating Systems Architectures**:
   For the web approach this will be a client-server system, where the client will handle HTML/CSS/JavaScript, and the server can follow a multi-tiered approach consisting of a Presentation Tier, Application Tier, and Data Tier.

3. **Storage Management**:
   Should be a combination of a relational database (RDBMS) and cloud storage. This will mean that the RDBMS will handle the structured data like user information, score, and game save state, and the cloud storage will handle the unstructured data like images and media files.

4. **Memory Management**: Modern web browsers include automated memory management in the form of garbage collection. This means memory will be allocated and deallocated for you, automatically preventing memory leaks and keeping memory usage efficient.

5. **Distributed Systems and Networks**:

Gameplay should exploit a distributed system with network facilities, such as a centralized server or even just a cloud service, that enables real-time updates, discussion, and connection between different players with different games / platforms. The game should be able to handle circumstances such as low bandwidth or momentary network disruptions and correctly recover from any resulting errors.

6. **Security**:

Security should be added to ensure that user information can be shared between multiple computers and the website. This would include secure connection support as well as username / password based authentication and protection of information. Additionally, information should be encrypted both on the server and on the local machine.